

Glossário de Termos — Aula 01

Git Fundamentos e Repositório Local

Este glossário reúne os termos e conceitos apresentados na Aula 01. Aqui você encontrará as definições fundamentais sobre controle de versão, Git e o fluxo de trabalho em repositório local. Todos os termos são essenciais para acompanhar as próximas aulas do módulo.

Controle de Versão

Controle de Versão (Version Control)

Sistema que registra alterações em arquivos ao longo do tempo, permitindo recuperar versões anteriores, comparar mudanças e colaborar com outras pessoas sem perder trabalho.

Analogia: Pense no "Ctrl+Z infinito": você pode voltar a qualquer ponto da história do seu projeto, não apenas ao passo anterior.

VCS Centralizado (CVCS)

Modelo de controle de versão onde existe um único servidor central que armazena todo o histórico. Exemplos: SVN, CVS. Se o servidor cair, ninguém consegue trabalhar.

Analogia: É como uma biblioteca com um único exemplar de cada livro — se ela fechar, ninguém lê.

VCS Distribuído (DVCS)

Modelo onde cada desenvolvedor possui uma cópia completa do repositório, incluindo todo o histórico. Git é o DVCS mais popular. Permite trabalho offline e maior segurança contra perda de dados.

Analogia: Cada pessoa tem a biblioteca inteira em casa — mesmo sem internet, você consegue consultar qualquer livro.

Git — Conceitos Fundamentais

Git

Sistema de controle de versão distribuído, criado por Linus Torvalds em 2005 para gerenciar o desenvolvimento do kernel Linux. É gratuito, open source e o padrão da indústria.

GitHub

Plataforma de hospedagem de repositórios Git na nuvem. Adiciona funcionalidades como Pull Requests, Issues, Actions (CI/CD) e colaboração em equipe. Git é a ferramenta; GitHub é o serviço.

Analogia: Git é o motor do carro; GitHub é a estrada onde você dirige e encontra outros motoristas.

Repositório (Repository / Repo)

Pasta do projeto que contém todos os arquivos e o histórico completo de alterações, armazenado na pasta oculta .git/. É o "banco de dados" do seu projeto.

```
git init → Cria um repositório na pasta atual
```

Diretório .git/

Pasta oculta criada pelo git init que contém toda a base de dados do repositório: objetos, referências, configurações e histórico. Nunca edite manualmente.

Configuração Inicial

git config

Comando para configurar o Git. As configurações podem ser em três níveis: --system (toda a máquina), --global (seu usuário) e --local (apenas o repositório atual).

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"
```

Identidade (user.name / user.email)

Nome e email que ficam gravados em cada commit. Obrigatórios antes do primeiro commit. Toda a equipe consegue ver quem fez cada alteração.

Branch Padrão (main)

O nome da branch principal criada automaticamente. Antigamente era "master"; a convenção moderna é "main".

```
git config --global init.defaultBranch main
```

Ciclo de Vida dos Arquivos

Working Directory (Diretório de Trabalho)

A pasta onde você edita seus arquivos normalmente. O Git monitora essa pasta para detectar mudanças, mas as alterações só são registradas quando você as adiciona à staging area.

Staging Area (Área de Preparação / Index)

Área intermediária onde você seleciona quais mudanças farão parte do próximo commit. Permite montar commits precisos, escolhendo exatamente o que incluir.

Analogia: É a bancada onde você organiza os itens antes de colocá-los na caixa de entrega (commit).

```
git add arquivo.txt → Adiciona à staging area
```

Commit

Registro permanente (snapshot) do estado dos arquivos na staging area. Cada commit tem um hash SHA-1 único, autor, data e mensagem descritiva. É o "ponto de salvamento" do projeto.

Analogia: Como tirar uma foto do projeto naquele momento — você pode voltar a qualquer foto a qualquer hora.

```
git commit -m "feat: adiciona página de login"
```

Tracked / Untracked

Arquivos tracked já foram adicionados ao Git em algum momento (Git conhece). Arquivos untracked são novos e nunca foram adicionados — o Git os ignora até que você faça git add.

Modified / Staged / Committed

Os três estados de um arquivo tracked: Modified (alterado mas não preparado), Staged (preparado para o próximo commit), Committed (salvo no repositório).

Comandos Essenciais

git init

Inicializa um repositório Git vazio na pasta atual, criando a pasta .git/.

```
git init meu-projeto
```

git status

Mostra o estado atual do repositório: quais arquivos foram modificados, quais estão na staging area e quais não estão sendo rastreados.

```
git status ou git status -s (formato curto)
```

git add

Move arquivos do working directory para a staging area. Pode adicionar arquivos específicos ou todos de uma vez.

```
git add arquivo.txt | git add . (adiciona tudo)
```

git commit

Grava as mudanças da staging area como um novo commit no histórico do repositório.

```
git commit -m "mensagem do commit"
```

git log

Exibe o histórico de commits do repositório, do mais recente ao mais antigo. Mostra hash, autor, data e mensagem.

```
git log --oneline --graph (formato compacto com grafo)
```

git diff

Mostra as diferenças entre arquivos. Sem argumentos, compara working directory com staging area. Com --staged, compara staging com último commit.

```
git diff | git diff --staged
```

git restore (moderno)

Comando moderno que substitui git checkout para restaurar arquivos. Desfaz mudanças no working directory ou remove da staging area.

```
git restore arquivo.txt → Desfaz mudanças  
git restore --staged arquivo.txt → Remove da staging
```

.gitignore e Conventional Commits

.gitignore

Arquivo na raiz do repositório que lista padrões de arquivos e pastas que o Git deve ignorar (não rastrear). Essencial para excluir `node_modules/`, `.env`, arquivos compilados, etc.

```
node_modules/  
*.class  
.env  
dist/
```

Conventional Commits

Convenção de mensagens de commit padronizada: `tipo(escopo): descrição`. Facilita a leitura do histórico, geração automática de changelogs e versionamento semântico.

```
feat: adiciona autenticação OAuth  
fix(api): corrige timeout na rota /users  
docs: atualiza README com instruções de deploy
```

Tipos de Commit (feat, fix, docs, style, refactor, test, chore)

feat = nova funcionalidade, fix = correção de bug, docs = documentação, style = formatação, refactor = refatoração sem mudar comportamento, test = testes, chore = tarefas de manutenção.